vLLM Runtime - Implementation Checklist

This checklist tracks the implementation of recommendations from the vLLM runtime audit.

✓ Completed (Audit Phase)

- [x] Map runtime execution chain
- [x] Audit all vLLM references
- [x] Verify provider configuration
- [x] Create audit script
- [x] Create smoke test suite
- [x] Document architecture
- [x] Create quick start guide
- [x] Generate executive summary

🔄 High Priority Recommendations

1. Add Logging to VLLMRuntime Internal Fallback

Status: ⏳ Pending
Priority: High
Effort: Low (30 minutes)
File: `src/ai_karen_engine/inference/vllm_runtime.py`

Current Code (Lines 100-106):

```
def generate(self, prompt: str, **kwargs: Any) -> str:
    if not self.base_url:
        return self._fallback_text(prompt, **kwargs)
    try:
        return self._provider.generate_text(prompt, **kwargs)
    except Exception:
        return self._fallback_text(prompt, **kwargs)  # ⚠ Silent
fallback
```

Recommended Change:

```
def generate(self, prompt: str, **kwargs: Any) -> str:
    if not self.base_url:
        logger.info("vLLM base_url not configured, using Transformers
fallback")
        return self._fallback_text(prompt, **kwargs)
    try:
        return self._provider.generate_text(prompt, **kwargs)
    except Exception as e:
```

```
        logger.warning(
            f"vLLM generation failed, using Transformers fallback: {e}",
            extra={"provider": "builtin_vllm", "fallback":
"transformers"}
        )
        return self._fallback_text(prompt, **kwargs)
```

Also Update Stream Method (Lines 111-119):

```
def stream(self, prompt: str, **kwargs: Any) -> Iterator[str]:
    if not self.base_url:
        logger.info("vLLM base_url not configured, using Transformers
fallback for streaming")
        yield from self._fallback_runtime.stream(prompt, **kwargs)
        return
    try:
        yield from self._provider.stream_generate(prompt, **kwargs)
        return
    except Exception as e:
        logger.warning(
            f"vLLM streaming failed, using Transformers fallback: {e}",
            extra={"provider": "builtin_vllm", "fallback":
"transformers"}
        )
        yield from self._fallback_runtime.stream(prompt, **kwargs)
```

Verification:

```
# After implementation, check logs for fallback events
grep "vLLM.*fallback" logs/karen_api.log
```

PROF

---

2. Add vLLM Smoke Tests to CI

Status: ⏳ Pending
Priority: High
Effort: Medium (1-2 hours)
Files: `.github/workflows/ci.yml` or equivalent CI config

Implementation:

1. **Add CI Job:**

```
# .github/workflows/ci.yml
jobs:
  vllm-smoke-tests:
```

```
    name: vLLM Smoke Tests
    runs-on: ubuntu-latest
    if: ${{ vars.VLLM_SERVER_AVAILABLE == 'true' }}

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest httpx

      - name: Run vLLM smoke tests
        env:
          KAREN_RUN_VLLM_SMOKE: "1"
          KAREN_VLLM_BASE_URL: ${{ secrets.VLLM_BASE_URL }}
          KAREN_VLLM_MODEL: ${{ secrets.VLLM_MODEL }}
        run: |
          pytest tests/integration/test_vllm_smoke.py -v --tb=short
```

2. **Add GitHub Secrets:**
   - `VLLM_BASE_URL`: vLLM server endpoint
   - `VLLM_MODEL`: Model name to test
3. **Add GitHub Variable:**
   - `VLLM_SERVER_AVAILABLE`: Set to `true` when vLLM server is available

PROF

Verification:

```
# Test locally first
KAREN_RUN_VLLM_SMOKE=1 \
KAREN_VLLM_BASE_URL=http://localhost:8001/v1 \
pytest tests/integration/test_vllm_smoke.py -v
```

---

3. Add vLLM Health Diagnostics Endpoint

Status: ⏳ Pending
Priority: High
Effort: Medium (2-3 hours)
File: `src/ai_karen_engine/api_routes/monitoring/health.py`

Implementation:

```
@router.get("/health/providers/vllm")
async def vllm_provider_health() -> Dict[str, Any]:
    """
    Detailed vLLM provider health check.

    Returns:
        Comprehensive health status including:
        - Provider configuration
        - Server connectivity
        - Model availability
        - Generation capability
        - Streaming support
    """
    from ai_karen_engine.inference.vllm_runtime import VLLMRuntime
    import time

    start_time = time.time()

    try:
        runtime = VLLMRuntime.get_instance()

        # Basic health check
        health = runtime.health_check()

        # Test actual generation
        generation_ok = False
        generation_error = None
        generation_latency = None

        try:
            gen_start = time.time()
            test_response = runtime.generate(
                "test",
                max_tokens=5,
                temperature=0.1
            )
            generation_latency = (time.time() - gen_start) * 1000
            generation_ok = bool(test_response and len(test_response) >
0)
        except Exception as e:
            generation_error = str(e)

        # Test streaming
        streaming_ok = False
        streaming_error = None

        try:
            chunks = list(runtime.stream("test", max_tokens=5))
            streaming_ok = len(chunks) > 0
        except Exception as e:
```

```
            streaming_error = str(e)

        response_time = (time.time() - start_time) * 1000

        return {
            "provider": "builtin_vllm",
            "enabled": True,
            "healthy": health.get("healthy", False) and generation_ok,
            "base_url": runtime.base_url,
            "default_model": runtime.model,
            "mode": health.get("mode", "vllm"),
            "checks": {
                "server_reachable": health.get("healthy", False),
                "models_endpoint_ok": health.get("models_available",
False),
                "generation_test_ok": generation_ok,
                "streaming_test_ok": streaming_ok,
            },
            "latency": {
                "health_check_ms": response_time,
                "generation_ms": generation_latency,
            },
            "errors": {
                "generation_error": generation_error,
                "streaming_error": streaming_error,
                "last_error": health.get("error"),
            },
            "capabilities": {
                "streaming_supported": True,
                "embeddings_supported": False,
                "chat_completion": True,
                "text_generation": True,
            },
            "details": health,
        }

    except Exception as e:
        return {
            "provider": "builtin_vllm",
            "enabled": True,
            "healthy": False,
            "error": str(e),
            "checks": {
                "server_reachable": False,
                "models_endpoint_ok": False,
                "generation_test_ok": False,
                "streaming_test_ok": False,
            },
        }
```

PROF

Verification:

```
# Test the endpoint
curl -sS http://localhost:8000/api/health/providers/vllm | jq
```

📋 Medium Priority Recommendations

4. Document vLLM Server Setup

Status: ⏳ Pending
Priority: Medium
Effort: Medium (2-3 hours)

Create: `docs/VLLM_SERVER_SETUP.md`

Contents:

- Installation instructions
- Model loading guide
- Configuration examples
- Docker setup
- Kubernetes deployment
- Troubleshooting

---

5. Add Chat Endpoint Contract Tests

Status: ⏳ Pending
Priority: Medium
Effort: Medium (2-3 hours)

Create: `tests/integration/test_chat_vllm_contract.py`

PROF

Test Cases:

- Explicit vLLM routing
- Fallback to vLLM
- Metadata structure validation
- Streaming contract
- Error handling

---

6. Enhance UI Provider Display

Status: ⏳ Pending
Priority: Medium
Effort: High (4-6 hours)

Files:

- `src/ui_launchers/Karen-AI-Theme/src/components/ChatMessage.tsx`
- `src/ui_launchers/Karen-AI-Theme/src/lib/types.ts`

Features:

- Show actual vs requested provider
- Display fallback indicators
- Show response source (live vs static)
- Display latency metrics

🕰️ Low Priority Recommendations

7. Cleanup Legacy llama.cpp References

Status: ⏳ Pending
Priority: Low
Effort: Medium (2-3 hours)

Action: Audit and document all llama.cpp references

---

8. Performance Benchmarks

Status: ⏳ Pending
Priority: Low
Effort: High (4-6 hours)

Create: `tests/performance/test_vllm_benchmarks.py`

---

9. Multi-Model Support

Status: ⏳ Pending
Priority: Low
Effort: High (6-8 hours)

Enhancement: Allow selecting specific vLLM models dynamically

---

📊 Progress Tracking

Completed: 8/17 (47%)
High Priority Remaining: 3
Medium Priority Remaining: 3
Low Priority Remaining: 3

🎯 Next Sprint Goals

1. ✅ Complete High Priority items (1-3)

2. 📝 Document vLLM server setup (4)
3. 🧪 Add contract tests (5)

📝 Notes

- All audit deliverables are complete and ready for use
- Audit script can be run anytime: `./scripts/audit_runtime_vllm.sh`
- Smoke tests are ready: `KAREN_RUN_VLLM_SMOKE=1 pytest tests/integration/test_vllm_smoke.py`
- Documentation is comprehensive and up-to-date

Last Updated: 2026-04-27
Next Review: After implementing High Priority items